# Predicción de la demanda horaria de bicicletas compartidas (Bike Sharing Demand)

**Proyecto Integrador, Parte I (EC2) · Metodología CRISP-DM** Equipo: Joan Marcelo Riveros Soria, Alejandro Miguel Pardo Romano y Jose Andres Manzaneda Manzaneda · Dataset: UCI *Bike Sharing Dataset* (Capital Bikeshare, Washington D.C.), fecha de acceso 17/10/2026.

> Las cifras de este informe provienen de código ejecutado en los notebooks del repositorio. En cada caso se indica el notebook o la ficha de `reports/decisiones.md` de donde salen.

## Resumen

Se construyó un modelo que predice la demanda por hora de bicicletas compartidas en toda la ciudad para el día siguiente (24 horas), con el pronóstico emitido a las 00:00. El modelo final es un LightGBM entrenado sobre el logaritmo de la demanda (`log1p`). Sus entradas son el calendario, el clima y la demanda observada hace 1, 7 y 14 días a la misma hora (`lag_24`, `lag_168` y `lag_336`).

| Evaluación | RMSLE | RMSE | MAE | $R^2$ |
|---|---|---|---|---|
| Validación temporal (solo train, 5 folds) | 0.3732 ± 0.0566 | | | |
| Test, jul–dic 2012 (una sola evaluación) | 0.3505 | 69.99 | 43.11 | 0.8985 |
| Baseline de calendario en test | 0.5999 | 150.35 | 107.21 | 0.5316 |

En el test el modelo reduce el error del baseline en 41.6 %, y la validación no fue optimista: el test dio un RMSLE incluso menor. También tiene límites. Tiende a subestimar la demanda (−6.9 % en el total diario), falla más con clima adverso, en feriados y de madrugada, y depende del historial: sin los lags predice un 85 % menos. Además, en el verano del test los lags no mejoraron al modelo sin ellos (0.3463). La validación no podía anticiparlo porque ningún fold cubría julio ni agosto.

## 1. Comprensión del negocio y formulación

**Problema.** La demanda horaria de bicicletas compartidas depende del clima, de la rutina laboral y de la estación del año. Anticiparla permite planificar la flota y su redistribución.

**Tipo de problema y target.** Es un problema de regresión sobre datos tabulares con orden temporal. El target es `cnt`, la demanda total de la ciudad por hora. Como el dato no está desglosado por estación de bicicletas, no se pueden sacar conclusiones por estación.

**Horizonte.** Se predice el día siguiente, con el pronóstico emitido a las 00:00. A esa hora ya se conoce la demanda de ayer y de las semanas anteriores, por lo que se pueden usar los lags de 24, 168 y 336 horas. El clima que entra al modelo sería un pronóstico y no el valor observado; su efecto se prueba en la sección 10.

**Criterio de éxito.** La métrica principal es el RMSLE, que mide el error en escala logarítmica, es decir, en términos relativos, y a igual error absoluto castiga más la subestimación. Como el modelo se entrena sobre `log1p(cnt)` con error cuadrático, optimiza directamente esta métrica. Las métricas secundarias son MAE, RMSE y $R^2$, calculadas en bicicletas.

Como umbral de éxito se propone mejorar al menos 30 % el RMSLE de validación del baseline de calendario (0.5585), es decir, llegar a un RMSLE de validación menor o igual a 0.3910. El modelo final lo cumple: 0.3732 en validación (33.2 % menos) y 0.3505 en el test frente a 0.5999 del baseline (41.6 % menos). Hay que aclarar que este umbral se fijó después de conocer los resultados de validación y del test; en los Sprints 1 a 3 solo se usó el baseline como referencia, sin un porcentaje definido. Por eso se presenta como un criterio de referencia y no como una meta fijada de antemano, y el grupo puede mantenerlo o ajustarlo.

**Riesgos.** (1) Fuga de variables: `casual` y `registered` suman `cnt`, así que no pueden ser entradas. (2) Fuga temporal: no se usa K-Fold aleatorio. (3) El crecimiento de la demanda entre 2011 y 2012, que los árboles no extrapolan.

## 2. Comprensión de los datos

**Fuente y variante.** `hour.csv` tiene 17 379 filas y 17 columnas, de 731 días (2011-01-01 a 2012-12-31). Se usan todas las filas, sin submuestreo. La cita obligatoria es Fanaee-T y Gama (2013), *Event labeling combining ensemble detectors and background knowledge*, Progress in Artificial Intelligence, doi:10.1007/s13748-013-0040-3.

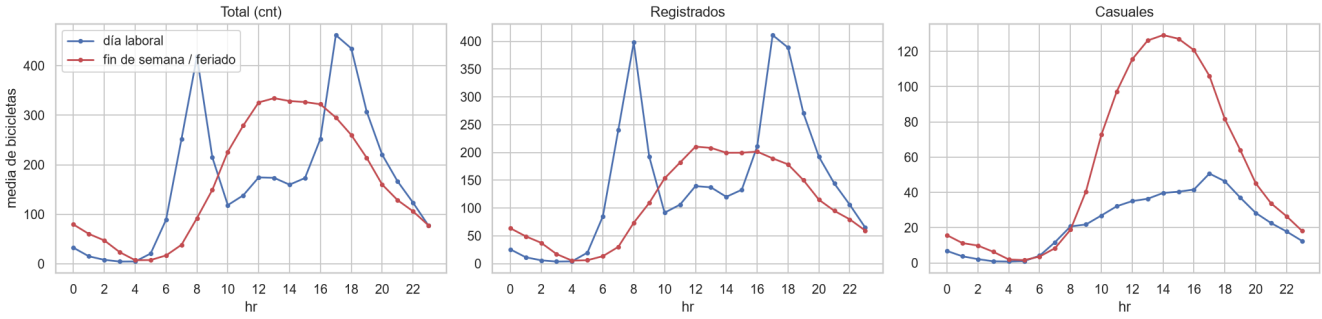
**Variables.** `season`, `yr`, `mnth`, `hr`, `holiday`, `weekday`, `workingday`, `weathersit` (de 1, despejado, a 4, lluvia o nieve intensa), `temp`, `atemp`, `hum` y `windspeed` (normalizadas dividiendo entre 41, 50, 100 y 67), `casual` y `registered` (excluidas como entrada) y `cnt` (el target). `instant` y `dteday` en bruto no se usan como entrada.

**Partición.** Train: 2011-01-01 a 2012-06-30 (13 128 horas, 74.8 %; 13 003 con registro). Test: 2012-07-01 a 2012-12-31 (4 416 horas; 4 376 con registro), que se evalúa una sola vez. Dentro de train se valida con `TimeSeriesSplit` de ventana expansiva, con 5 folds de 1 460 horas (unos 2 meses) y el primer fold entrenando con unos 8 meses. Las ventanas de validación van del 2011-08-31 al 2012-06-30, por lo que ningún fold valida julio ni agosto (notebook 03).

### 2.1 Calidad de los datos (notebook 01)

| Hallazgo | Dato verificado | Decisión |
|---|---|---|
| Horas ausentes | 165 de 17 544 (0.94 %); 84 en las horas 2–4; ene-2011 (56), oct-2012 (36); racha máxima de 36 horas, mediana de 1 | Reindexar a las 17 544 horas antes de cualquier `shift`; target vacío y excluido del ajuste y de la métrica |
| `windspeed = 0` | 2 180 filas (12.5 %); el siguiente valor distinto es 0.0896 (≈ 6 km/h) | Posible umbral del sensor (hipótesis); la prueba con LightGBM no mostró efecto, así que se mantiene el valor |
| `hum = 0` | 22 filas, todas el 2011-03-10 | Imputar con la mediana de train |
| `weathersit = 4` | 3 filas (0.017 %), las tres en train | Fusionar con la categoría 3 |
| `temp` frente a `atemp` | r = 0.9877; VIF 68.65 y 68.78 | Usar `temp_c` y `temp_diff = atemp_c − temp_c`: VIF 1.40 y 1.52 |

## 3. Análisis exploratorio (solo con train; notebook 01)




1. **La dinámica horaria depende del tipo de día.** En días laborables hay dos picos, a las 8:00 (≈ 418 bicicletas/hora) y a las 17:00 (≈ 462). En fines de semana y feriados hay una sola campana, con máximo a las 13:00 (≈ 334). Los usuarios registrados siguen los picos laborales y los casuales dominan el fin de semana (32.8 % de los viajes frente a 13.4 % en días laborables). Por eso se creó la categoría combinada `hr_workingday` (48 niveles) en lugar de multiplicar las dos variables.
2. **Distribución del target.** `cnt` tiene asimetría positiva (1.29; mediana 124 frente a media 167.7). Con `log1p` la asimetría pasa a −0.81: baja en valor absoluto pero no desaparece. La razón principal de usar `log1p` es alinear el entrenamiento con el RMSLE.
3. **Tendencia.** Comparando el mismo semestre (ene–jun), la demanda media pasa de 123.4 a 215.2 bicicletas/hora (+74 %; registrados +78 %, casuales +61 %). La razón entre 2012 y 2011 baja de 2.53 en marzo a 1.41 en junio. Con estos datos no se puede afirmar qué ocurre en la segunda mitad de 2012.
4. **Clima.** La demanda media es 183.4 con clima 1, 152.0 con clima 2 y 93.9 con clima 3 (8.7 % de las filas de train). La temperatura es el factor más fuerte: de 48.6 bicicletas con 5 °C o menos a 289.7 por encima de 30 °C.

5. **Evento Sandy (2012-10-29/30).** Cae en el test y el 29-oct solo hay una hora registrada. Antes de usar el test solo se documentó su estructura.

---

## 4. Preparación de los datos (`src/data_loader.py`, `src/features.py`)

- **Serie continua.** `load_data()` reindexa a las 17 544 horas y reconstruye el calendario de las horas ausentes; el clima y el target quedan vacíos, sin inventar valores.
- **Preprocesador.** Un `Pipeline` aplica primero transformaciones que no aprenden nada (`weathersit = 4` pasa a 3, `temp_c`, `temp_diff`, `hr_workingday`) y luego un `ColumnTransformer` con One-Hot de `season`, `weathersit` y `hr_workingday`, imputación por la mediana de train y escalado solo para los modelos lineales. Todo lo que se aprende se ajusta únicamente con train.
- **Lags.** `agregar_lags` crea `lag_24`, `lag_168` y `lag_336` sobre la serie completa reindexada (por eso importa la reindexación). Cada lag solo mira el pasado de esa hora.
- **Control de fuga.** `casual`, `registered`, `instant` y `dteday` en bruto no entran; los clusters se ajustan sin el target; el test no se usa hasta la evaluación final.

---

## 5. Modelado supervisado

### 5.1 Modelos y técnicas elegidos, y qué aportó cada uno

Antes de los resultados, esta sección resume qué hace cada modelo o técnica, por qué se eligió y qué aportó. Las cifras son el RMSLE medio de validación (solo train, 5 folds), salvo que se indique otra cosa.

**Modelos**

| Modelo | Qué hace | Por qué se eligió | Qué aportó |
|---|---|---|---|
| Baseline: media de `cnt` por hora y tipo de día | Predice el promedio histórico de cada combinación de hora y `workingday` | Es la referencia mínima que cualquier modelo debe superar | RMSLE 0.5585 en validación y 0.5999 en test; ya explica ≈ 62 % de la varianza |
| Regresión lineal sobre variables crudas | Ajusta una recta sobre las variables sin transformar | Segundo baseline, muy simple | RMSLE 1.1055: muestra que la relación con la hora no es lineal (tiene dos picos) |
| Ridge con splines de la hora × temperatura | Regresión lineal con una penalización que reduce los coeficientes grandes; los splines representan curvas suaves | Modelo lineal interpretable visto en clase | RMSLE 0.4885, 12.5 % mejor que el baseline |
| Random Forest | Promedia muchos árboles de decisión entrenados con muestras distintas | Captura relaciones no lineales sin mucha preparación | RMSLE 0.4826, parecido al de Ridge |
| LightGBM | Entrena árboles pequeños uno tras otro, y cada uno corrige los errores de los anteriores | Suele funcionar bien con datos tabulares y aprende interacciones sin diseñarlas a mano | RMSLE 0.4549 con ajuste de hiperparámetros y 0.3732 con tres lags; es el modelo final |
| AutoML (FLAML) | Prueba modelos e hiperparámetros automáticamente dentro de un tiempo fijo | Sirve de comparación con el trabajo manual | RMSLE 0.4405 sin lags, mejor que el manual (0.4549); encontró árboles muy pequeños que el ajuste manual no había explorado |

**Técnicas**

| Técnica | Qué hace | Por qué se eligió | Qué aportó |
|---|---|---|---|
| Variable `hr_workingday` | Combina la hora y el tipo de día en una sola categoría | El perfil horario es distinto en días laborables y fines de semana | En Ridge, el RMSLE bajó de 1.0992 (hora entera, sin la categoría) a 0.4930 (con ella), según la ficha 8 |
| Transformación `log1p` del target | Entrena sobre el logaritmo de la demanda y revierte con `expm1` | Alinea el entrenamiento con el RMSLE y reduce la asimetría (de 1.29 a −0.81) | Permite optimizar la métrica principal directamente |

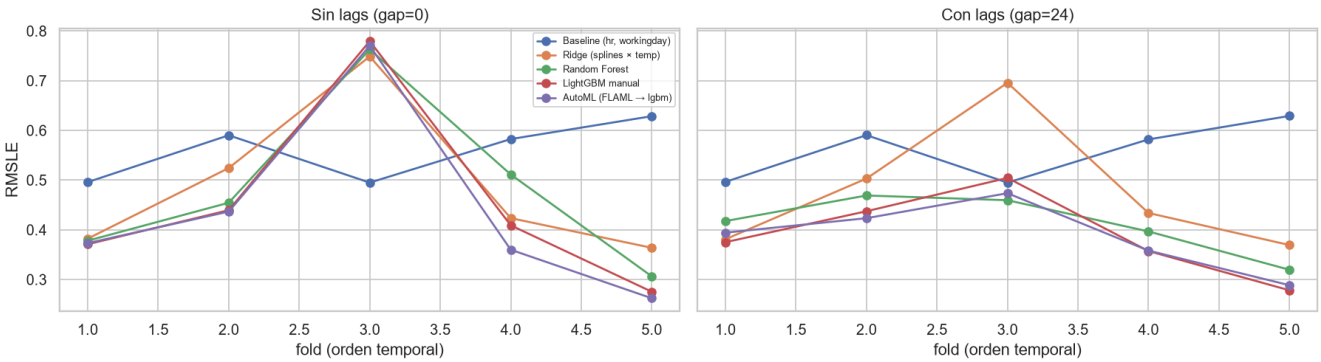
| Técnica | Qué hace | Por qué se eligió | Qué aportó |
|---|---|---|---|
| Lags de 24, 168 y 336 horas | Añaden como entradas la demanda de hace 1, 7 y 14 días a la misma hora | La demanda de la misma hora en días y semanas anteriores es muy informativa e incorpora el nivel reciente | LightGBM pasó de 0.4562 a 0.3732 (18.2 % menos) y el fold 3 bajó de 0.7786 a 0.4117 |
| `TimeSeriesSplit` | Valida respetando el orden de las fechas | Evita que el modelo vea el futuro al validar | La validación (0.3732) no sobreestimó el rendimiento: el test dio 0.3505 |
| K-Means con PCA y UMAP | Agrupa horas con condiciones parecidas y proyecta los datos para verlos | Buscar perfiles de operación sin usar el target | k = 4, con estructura débil (silueta 0.18); sirve para segmentar el error por tipo de hora |
| SHAP y permutación | Miden cuánto usa el modelo cada variable | Entender en qué se apoya el modelo | La hora aporta entre 23 y 25 % de la importancia SHAP total y los lags son muy importantes (`lag_168` entre 22 y 28 %, `lag_24` ≈ 8 %) |

## 5.2 Resultados de los baselines y modelos (validación temporal, solo train)

| Modelo | RMSLE medio ± std (5 folds) | Fuente |
|---|---|---|
| Baseline: media de `cnt` por `(hr, workingday)` | 0.5585 ± 0.0601 | notebook 03 |
| Regresión lineal sobre variables crudas | 1.1055 ± 0.0951 | notebook 03 |
| Ridge (splines de la hora × temperatura) | 0.4885 ± 0.1583 | notebook 03b, ficha 9 |
| Random Forest con ajuste de hiperparámetros | 0.4826 ± 0.1748 | notebook 05 (ver nota) |
| LightGBM con ajuste de hiperparámetros | 0.4549 ± 0.1918 | notebook 03, ficha 4 |
| AutoML (FLAML, que eligió un LightGBM) | 0.4405 ± 0.1946 | notebook 04, ficha 14 |

Nota sobre Random Forest: las cifras de este informe (0.4826 sin lags y 0.4125 con lags) se calcularon en el notebook 05. El notebook 03c, ejecutado en otro equipo, da 0.4827 y 0.4124 (fichas 12 y 13). La diferencia de 0.0001 no cambia ninguna conclusión; el Random Forest no reproduce la cuarta cifra decimal entre máquinas.

- El baseline de calendario es fuerte. La regresión lineal cruda falla incluso dentro de train (RMSLE 1.0175) porque la hora tiene dos picos y no se comporta de forma lineal.
- En Ridge, la forma de codificar la hora solo importa cuando falta la categoría `hr_workingday`. Con ella, splines, One-Hot y sin/cos dan lo mismo (0.4930 a 0.4932). Lo que mejoró en los 5 folds fue multiplicar la hora por la temperatura. Lasso no mejoró.
- En LightGBM, las pruebas de preparación no cambiaron nada medible: quitar `yr` empeoró el RMSLE a 0.5370, y el tratamiento de `windspeed` y `hum` no tuvo efecto. El ajuste de hiperparámetros con `RandomizedSearchCV` y `TimeSeriesSplit` bajó el RMSLE de 0.4643 a 0.4549 y mejoró los 5 folds; la variación entre semillas fue de 0.0002. Las diferencias entre modelos (≈ 0.03) son menores que la desviación entre folds (0.16 a 0.19), por lo que se compararon fold a fold.
- Dos pruebas no se adoptaron: el objetivo Poisson (0.4473, peor en 2 folds) y el factor de *smearing* de Duan (φ ≈ 1.028; peor en 2 folds). Están en las fichas 5 y 6.
- El fold 3 concentra el error: valida de enero a marzo de 2012 con un train que solo tiene 2011, y ningún modelo había visto el nivel más alto de 2012.



### 5.3 Los lags y el horizonte (notebooks 05 y 06)

Con los lags el modelo conoce el nivel reciente de demanda, y eso corrige buena parte del error del fold 3. Resultados con `get_cv(gap = 24)`:

| Escenario | LightGBM manual | AutoML (config sin lags) |
|---|---|---|
| Sin lags | 0.4562 | 0.4417 |
| Semana: `lag_168` + `lag_336` | 0.3893 | 0.3863 |
| Día (00:00): `lag_24` + `lag_168` | 0.3905 | 0.3855 |
| Día (00:00): los tres lags | 0.3732 ± 0.0566 | 0.3738 ± 0.0673 |
| Día emitido a las 18:00, tres lags | 0.3823 | 0.3826 |
| Falla de `lag_24` (entrenado con los tres) | 0.3862 | 0.3985 |

- Los lags no obligan a un horizonte diario. El esquema semanal (`lag_168` + `lag_336`) casi iguala al diario con dos lags; con los tres lags, el diario supera al semanal en 0.0188 (IC 95 % por bootstrap de días [−0.0243, −0.0133], probablemente optimista).
- Con los tres lags, LightGBM manual empata en precisión con AutoML (diferencia −0.0020, IC 95 % [−0.0081, 0.0039]) y resiste mejor la falta de `lag_24`.
- Esta combinación no cumple la regla de adopción que el equipo se había fijado, porque mejora la media en 18.2 % pero empeora 2 de 5 folds. Se adoptó porque la ganancia es grande en el fold más difícil. La regla resulta conservadora con mejoras que dependen de un solo fold.

### 5.4 Selección del modelo final

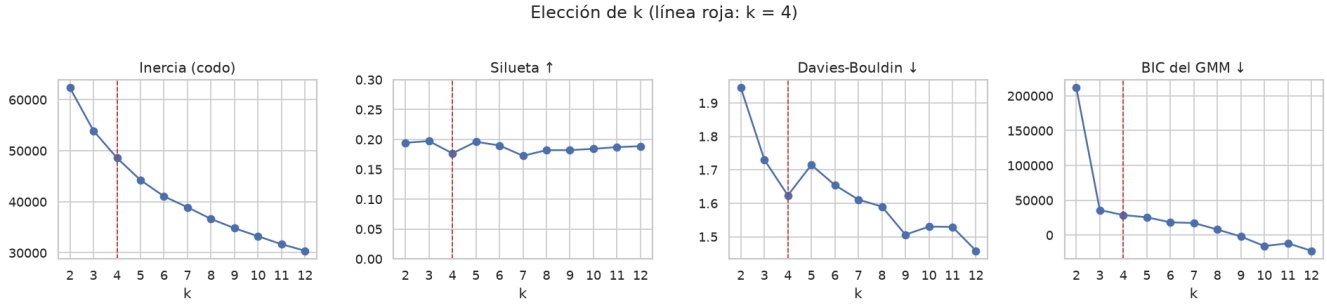
Se eligió LightGBM manual con los tres lags, usando solo la validación y antes de ver el test. Es el mejor y el más estable en validación, resiste mejor la falta de `lag_24`, empata con AutoML y no depende de una búsqueda que no se puede reproducir. Se descartaron Random Forest, Ridge y el baseline: con lags obtienen 0.4125, 0.4766 y 0.5586, frente a 0.3905 de LightGBM con dos lags (la misma condición de comparación) y 0.3732 con tres (fichas 20 y 21).

---

## 6. Aprendizaje no supervisado (notebook 02, Pardo)

Se aplicó K-Means sobre `hr_sin`, `hr_cos`, `temp`, `hum`, `windspeed` y `workingday`, ajustado solo con train. Los índices internos no deciden cuántos grupos usar (la silueta está entre 0.17 y 0.20 para cualquier k, y el BIC del GMM no tiene mínimo). Se eligió k = 4 por estabilidad (ARI de 0.997 entre semillas y de 0.955 entre ajustar con 2011 y con todo train) y porque los grupos se pueden interpretar: tarde cálida y seca (demanda media 278), mañana laborable (138), no laborable (125) y noche laborable (118).

La estructura es débil: los clusters explican el 17.5 % de la varianza de `log1p(cnt)`, frente al 81 % que explican la hora y `workingday`. Por eso se usan para segmentar el error del modelo y no como variable de entrada. Ningún cluster representa el clima adverso porque el clima no entra en el agrupamiento. PCA reparte la varianza entre varias componentes (4 componentes para 79 %) y UMAP muestra dos bloques según `workingday`, con un gradiente continuo por hora.




---

## 7. AutoML como benchmark (notebook 04, José)

Se usó FLAML, que funciona con Python 3.13 y no necesita Java (PyCaret 3.3 exige Python 3.11 o anterior). Se le pasó el mismo `TimeSeriesSplit` de `get_cv()`, con tarea de regresión, métrica RMSE sobre `log1p(cnt)` (equivalente al

RMSLE), presupuesto de 300 s y semilla 42. Sin lags obtuvo 0.4405 frente a 0.4549 del mejor manual (mejor en 4 de 5 folds); con lags, 0.3876 frente a 0.3905. AutoML encontró una región que el ajuste manual no había explorado: árboles muy pequeños (`num_leaves = 4`, `n_estimators = 2 027`).

| Dimensión | LightGBM manual | AutoML (FLAML) |
|---|---|---|
| RMSLE en test (3 lags / sin lags) | 0.3505 / 0.3463 | 0.3389 / 0.3271 |
| RMSLE de validación (3 lags) | 0.3732 ± 0.0566 | 0.3738 ± 0.0673 |
| Búsqueda de hiperparámetros | ≈ 300 s, semilla fija | 302 s y 299 s; depende del tiempo de cómputo |
| Robustez si falta `lag_24` (validación) | 0.3862 | 0.3985 |
| Interpretabilidad | `LGBMRegressor` | Igual: el ganador es un `LGBMRegressor` |

La búsqueda de FLAML no se reproduce entre ejecuciones (la ficha 14 tuvo que corregirse porque sus números venían de una ejecución anterior). En cambio, el modelo ganador sí se reproduce a 6 decimales si se fija su configuración (ficha 15).

---

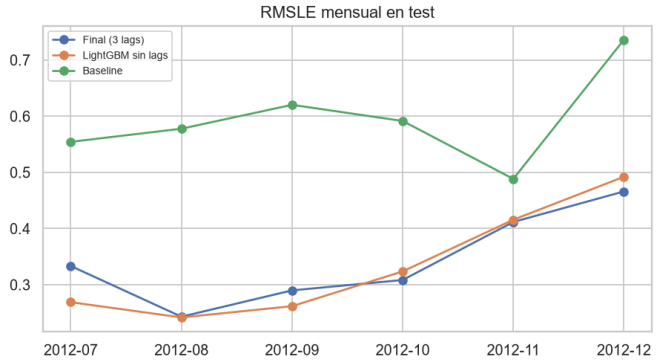
## 8. Evaluación final en el test (notebook 07)

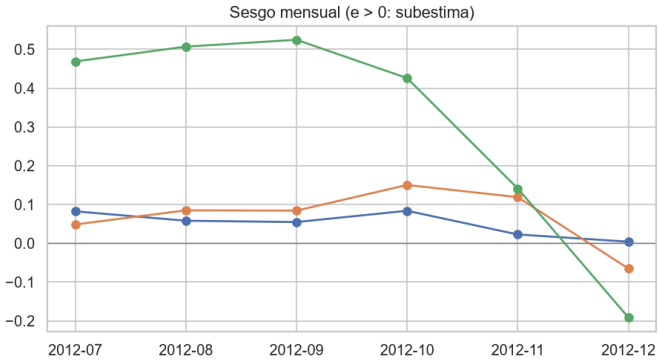
El modelo se reentrenó una vez con todo train y se evaluó una vez sobre las 4 376 horas con registro del test.

| Modelo (test) | RMSLE | RMSE | MAE | $R^2$ |
|---|---|---|---|---|
| Baseline | 0.5999 | 150.35 | 107.21 | 0.532 |
| Ridge (sin lags) | 0.3857 | 86.39 | 51.56 | 0.845 |
| Random Forest (sin lags) | 0.3812 | 78.91 | 50.87 | 0.871 |
| LightGBM sin lags | 0.3463 | 69.17 | 43.81 | 0.901 |
| AutoML (config sin lags, sin lags) | 0.3271 | 67.02 | 42.63 | 0.907 |
| LightGBM final (3 lags) | 0.3505 | 69.99 | 43.11 | 0.8985 |

El RMSLE del test (0.3505) es menor que el de validación (0.3732 ± 0.0566) y queda dentro del rango de los folds (0.2916 a 0.4311). Algunas explicaciones posibles, que no se probaron: el modelo ya conoce el nivel de 2012, el test tiene más demanda (el RMSLE tiende a ser menor con más bicicletas) y no contiene un cambio de nivel como el de enero a marzo de 2012.

Un resultado no esperado: los lags no mejoraron a LightGBM en el test (0.3505 frente a 0.3463 sin lags), y AutoML sin lags fue el mejor del test (0.3271). Los lags ayudan en noviembre y diciembre (0.412 y 0.466 frente a 0.416 y 0.492) y empeoran de julio a octubre (en julio, 0.333 frente a 0.269). No se cambió el modelo después de ver el test: hacerlo habría usado el test para decidir y dejado la cifra sin una estimación independiente. Es el riesgo que la validación no podía medir, porque ningún fold cubría el verano.



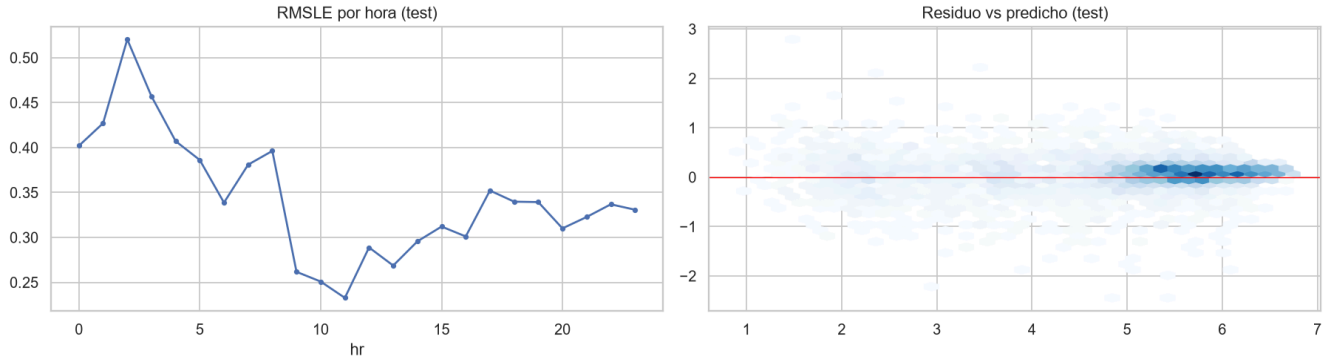




| Mes | jul | ago | sep | oct | nov | dic |
|---|---|---|---|---|---|---|
| RMSLE | 0.333 | 0.243 | 0.290 | 0.308 | 0.412 | 0.466 |
| Sesgo en log (> 0 subestima) | +0.082 | +0.058 | +0.055 | +0.084 | +0.023 | +0.004 |

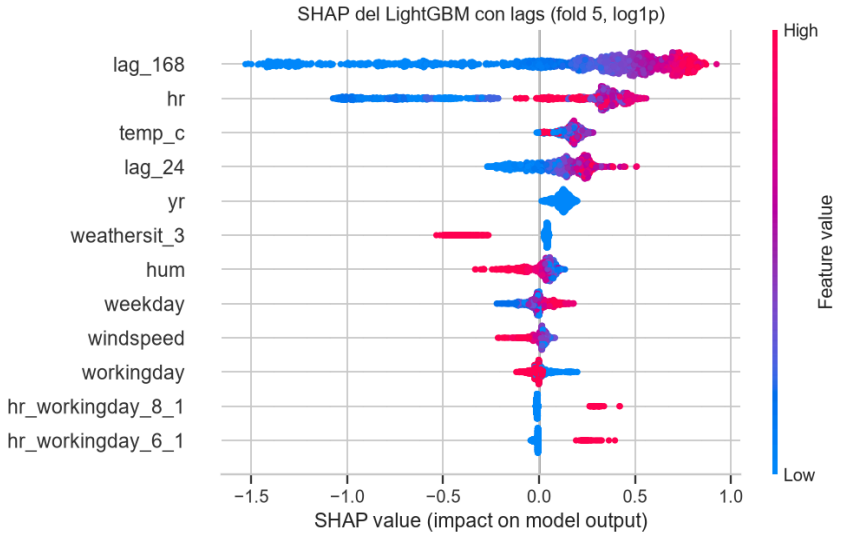
---

## 9. Análisis de errores e interpretabilidad (notebooks 05 y 07)

**Dónde falla** (test, modelo final). Con clima adverso (290 horas) el RMSLE es 0.654 y el modelo subestima (+0.182). En feriados (143 horas) es 0.664 y sobreestima (−0.088). A las 2:00 es 0.520. En días no laborables es 0.381, frente a 0.335 en laborables. Por cluster, el RMSLE es 0.223 en la tarde cálida y 0.339, 0.403 y 0.412 en los otros tres. En la validación (notebook 05, LightGBM con lags) el residuo no tiene dispersión constante: baja de 0.496 en las predicciones bajas a 0.189 en las altas. Por eso el RMSLE global está dominado por las horas de poca demanda.




**En qué se apoya el modelo** (SHAP y permutación en tres folds, notebook 05). Usa sobre todo la hora (entre 23 y 25 % de la importancia SHAP total, más 12 a 14 % de `hr × workingday`) y los lags (`lag_168` entre 22 y 28 %, `lag_24` ≈ 8 %). Después vienen el tipo de día, la temperatura, el clima y la humedad. Sin sus lags, el RMSLE de un fold sube entre 0.45 y 0.73. Las variables correlacionadas (`temp` y `atemp`, los lags) se reparten la importancia, por lo que se permutaron en grupos. Estos resultados describen cómo usa el modelo los datos, no relaciones de causa y efecto.




**Evento Sandy.** El 29-oct solo tiene una hora registrada y el 30-oct solo 11 (de las 13:00 a las 23:00). En esas 11 horas del 30-oct el modelo predice 1 606 bicicletas frente a 1 096 registradas (1.46 veces); la mayor diferencia es a las 19:00 (404 predichas frente a 115). Los lags tardan días en reflejar un evento sin precedente.

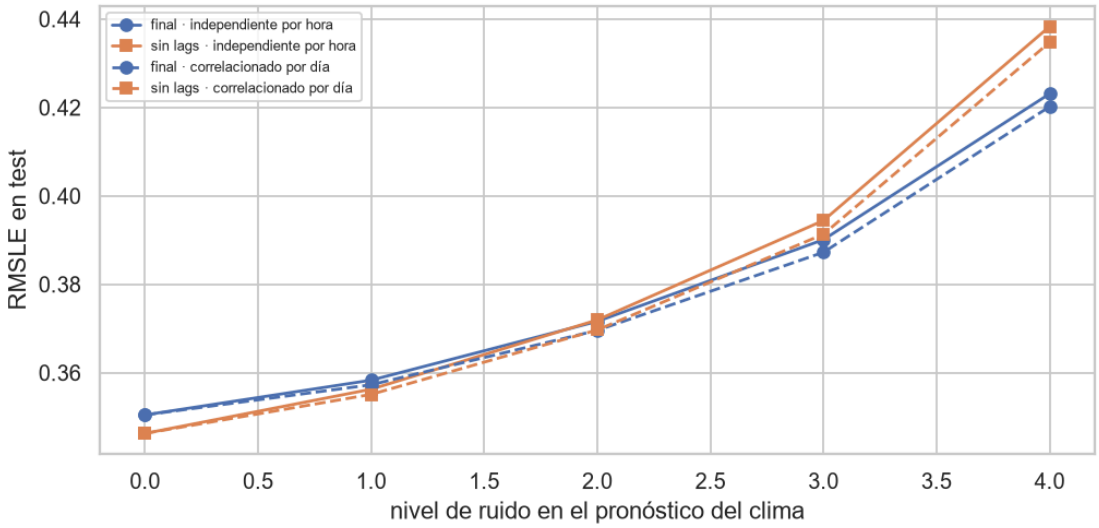
---

## 10. Validación operativa: ¿funcionaría en un caso real? (notebook 08, `src/predict.py`)

Para saber si el modelo serviría en un uso real se hicieron seis pruebas con el modelo final. La tabla indica cómo se hizo cada una y qué resultado dio.

| Prueba | Cómo se hizo | Resultado |
|---|---|---|
| Predicción de punta a punta | Se escribió `src/predict.py`, que recibe la fecha, el pronóstico de 24 horas y el historial, y devuelve las 24 predicciones. Se comparó con las predicciones del notebook 07 | Coinciden en 4 272 horas (diferencia máxima 5.7e-14). La función valida el formato de la entrada y rechaza historiales con datos del futuro y los 6 días con horas ausentes |

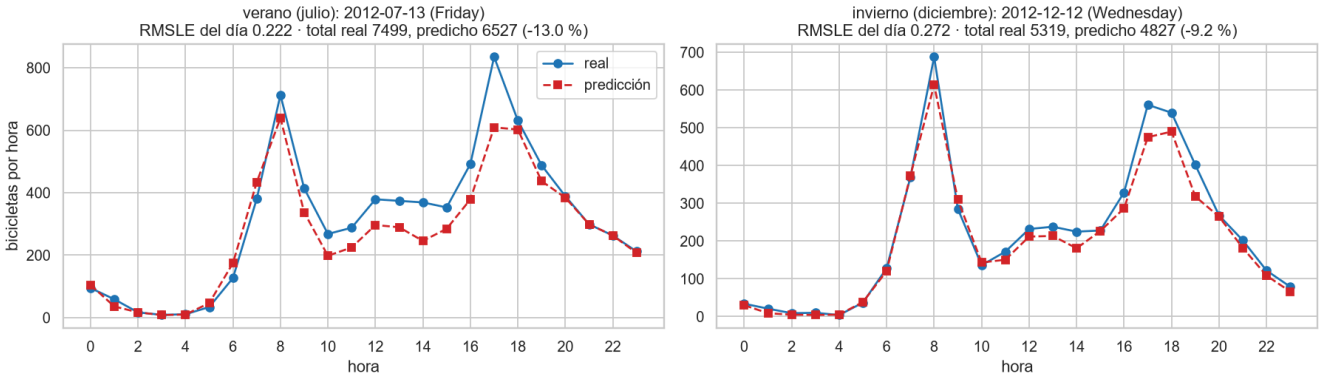
| Prueba | Cómo se hizo | Resultado |
|---|---|---|
| Ruido en el pronóstico del clima | Se perturbaron temperatura, humedad, viento y categoría del clima del test en cuatro niveles | En el nivel 2 (±2 °C, ±10 puntos de humedad, ±5 km/h y 10 % de cambios de categoría) el RMSLE sube de 0.3505 a 0.3716 (+6.0 %); en el nivel máximo llega a 0.4231 (+20.7 %). Sin lags el modelo se degrada más (+26.6 %). Los niveles son de referencia, no están calibrados con pronósticos reales |
| Simulación día a día | Se simuló usar el modelo cada día a las 00:00 durante 178 días del test | RMSLE diario: mediana 0.267, P90 0.475, máximo 0.916 (26-dic). El total del día se desvía con una mediana de 10.3 % (P90 25.7 %) y un sesgo de −6.9 %. Reentrenar cada mes baja el RMSLE a 0.3469 (−1.0 %) |
| Frecuencia de errores grandes | Se midió con qué frecuencia el error supera el 20 % (umbral propuesto) | Subestima más del 20 % en 23.2 % de las horas (16.1 % en horas pico) y sobreestima en 16.4 %. En horas pico el sesgo es de −50.3 bicicletas por hora |
| Casos límite | Se probaron entradas extremas: frío extremo, clima adverso, humedad de 100 % y falta de historial | Ninguno falla ni da valores negativos (máximo 834 bicicletas/hora). La predicción baja 45.9 % con frío extremo, 40.8 % con clima adverso y 31.7 % con humedad de 100 %. Sin ningún lag cae 84.8 % (6.7 % si falta solo `lag_24`) y `predict.py` emite un aviso en ese caso |
| Cambio entre train y test | Se compararon las distribuciones de las variables con el PSI | Cambian sobre todo la demanda (PSI 0.223), `lag_168` (0.264) y la temperatura (0.213); la humedad (0.070) y el viento (0.041) son estables |



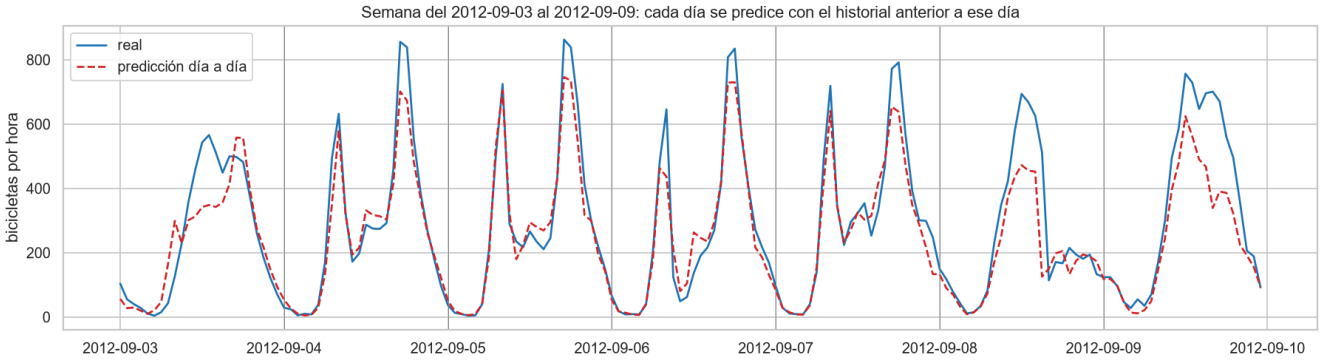

En conjunto, el modelo serviría en un uso real si se cumplen cuatro condiciones: contar con el pronóstico completo de las 24 horas, tener el historial reciente completo (si no, usar un modelo sin lags), dejar un margen de seguridad en horas pico y reentrenar periódicamente, porque la demanda crece.

### 10.1 Demostración de uso (notebook 09)

La demostración usa `src/predict.py` para mostrar cómo se emplea el modelo; no se usó para decidir nada. En los días reales el clima observado hace de pronóstico (el mejor caso posible, porque el dataset no trae pronósticos de 2012). Los días del test se eligieron con una regla fija, el día laborable sin feriado ni clima adverso más cercano a la mediana de su mes, y no por su resultado.




- **Dos días reales.** El viernes 2012-07-13 (verano) el RMSLE fue 0.222, con un total real de 7 499 frente a 6 527 predichas (−13.0 %). El miércoles 2012-12-12 (invierno) fue 0.272, con 5 319 frente a 4 827 (−9.2 %). El modelo reproduce los dos picos laborales y subestima en ambos días, igual que el sesgo medido en el test.
- **Escenario «mañana» sintético.** Con un pronóstico escrito a mano y un historial sintético de 14 días, la función devuelve las 24 predicciones (picos de ≈ 520 y ≈ 543 bicicletas a las 8:00 y 17:00), avisa de que la fecha (2013-04-16) queda fuera del rango de entrenamiento y rechaza con mensajes claros cinco pronósticos inválidos y un historial con horas del día a pronosticar. Es un ejemplo de uso, no una predicción real.
- **Un caso que falla a propósito.** Con el día de verano, sin los datos de ayer la predicción baja 2.6 %; sin ningún historial cae 82.4 % (de 6 527 a 1 150 bicicletas; el RMSLE del día pasa de 0.22 a 1.75) y `predict.py` emite un aviso.




- **Una semana de 7 días seguidos** (del 2012-09-03 al 09-09, la primera semana completa de septiembre, que incluye un feriado). El RMSLE fue 0.360 y el total semanal quedó 11.8 % por debajo de lo real. De martes a viernes el RMSLE diario está entre 0.19 y 0.29; falla en el feriado (0.58) y el fin de semana (totales −23.0 % y −27.2 %).

## 11. Limitaciones

1. Es la demanda total de la ciudad, no por estación.
2. El modelo necesita el historial reciente; sin él predice muy por debajo.
3. Ningún fold de validación cubre julio ni agosto, y los lags no ayudaron en el verano del test.
4. Los puntajes son algo optimistas: los hiperparámetros y los escenarios se eligieron sobre los mismos folds, y los intervalos por bootstrap solo recogen la variación entre días de las mismas cinco ventanas.
5. Las pruebas de ruido en el clima usan niveles de referencia no calibrados con pronósticos reales.
6. El modelo subestima de forma sistemática (−6.9 % en el total diario) por el crecimiento de 2012.
7. Los clusters son una hipótesis (silueta 0.18) y se ajustaron con todo train, sin usar el target.
8. No se probaron `LightGBM (casual + registered)` ni Optuna, que eran extras opcionales del plan.

## 12. Conclusiones y siguientes pasos

El LightGBM con tres lags reduce el error del baseline de calendario en 41.6 % en un test que no se usó para tomar decisiones, y la validación temporal anticipó razonablemente bien su rendimiento (0.3732 en validación frente a 0.3505 en test). Además de la cifra, el trabajo dejó un procedimiento ordenado: un mismo esquema de validación temporal para todos los modelos, fichas de decisión para cada prueba (también para las que no mejoraron), el

modelo elegido antes de ver el test, y los resultados desfavorables a la vista: los lags no ayudaron en el verano, el modelo falla sin historial y la búsqueda de AutoML no se reproduce exactamente.

La propuesta para la siguiente etapa se describe en la sección 13.

## 13. Plan de la siguiente etapa (Parte II)

Lo que sigue es una propuesta; el grupo debe decidir el alcance, las herramientas, el orden y las fechas. Cada línea parte de un resultado de esta etapa y se evaluaría con el mismo protocolo: validación temporal con `TimeSeriesSplit`, fichas de decisión y el test intacto hasta una evaluación final.

### 13.1 Qué resultados motivan cada línea

| Resultado de la Parte I | Qué implica para la siguiente etapa |
|---|---|
| Las diferencias entre modelos son menores que la desviación entre folds (LightGBM con tres lags: 0.3732 en validación y 0.3505 en test) | Todo modelo nuevo se compara fold a fold con el mismo protocolo, no por una sola cifra |
| El modelo depende de los lags (sin historial predice 84.8 % menos) y en el verano del test no ayudaron (0.3505 frente a 0.3463 sin lags) | Hace falta una forma de resumir el historial reciente y un modelo de respaldo para cuando falte |
| Los clusters tienen estructura débil (silueta 0.18) | Probar otras representaciones de las horas antes de agrupar |
| La demanda cambia (PSI 0.223 y 0.264) y el modelo subestima (−6.9 % en el total diario) | Monitorear la demanda y definir cuándo reentrenar |
| La búsqueda de AutoML no se reproduce, `models/` no está en Git y el registro de pruebas es manual (24 fichas) | Registrar y versionar de forma automática las ejecuciones, los datos y los modelos |

### 13.2 Líneas de trabajo

1. **Red neuronal para series de tiempo.** Se probaría una red secuencial (GRU o LSTM, o una convolución temporal) que reciba las últimas 336 horas de demanda y el clima de las 24 horas a pronosticar, y prediga las 24 horas a la vez. El primer paso es repetir el protocolo (`get_cv`, `log1p`, RMSLE) con una red pequeña y compararla con LightGBM (0.3732 en validación). Se adoptaría solo si cumple la regla de adopción y se evaluaría en el test una vez. Con 13 003 horas de entrenamiento es posible que la red no supere al árbol; si ocurre, también se documenta.
2. **Embeddings.** Se reemplazaría el One-Hot de `hr_workingday` por un embedding aprendido dentro de la red de la línea 1, y se probaría una representación del día completo (clima de 24 horas más calendario). Funcionaría si mejora el RMSLE o si produce grupos más estables que K-Means.
3. **Búsqueda de días parecidos y asistente sobre la documentación.** Un RAG clásico no encaja con datos numéricos, así que se proponen dos usos acotados. El primero busca días parecidos con los embeddings y usa la demanda media de esos días como variable o como explicación. El segundo es un asistente sencillo sobre el informe y las 24 fichas, que responda citando la ficha de origen. La recuperación de imágenes no aplica porque el dataset no tiene imágenes.
4. **Seguimiento, versionado e inferencia (MLOps).** Hoy las pruebas se registran a mano y solo el código está en Git. Se propone registrar parámetros, métricas y modelos de cada ejecución, versionar los datos y los modelos, y ofrecer el modelo como servicio con la misma validación de entrada de `predict.py`. Las herramientas (por ejemplo MLflow y DVC) están por decidir y por comprobar con Python 3.13. Estaría terminado cuando cualquier cifra del informe se pueda reproducir a partir de un identificador de ejecución.
5. **Demostración del sistema completo.** Un flujo que tome el pronóstico del clima, lo valide, calcule los lags desde el historial, ejecute el modelo y muestre las 24 predicciones junto a lo real. El primer paso es ejecutar `predecir_dia` cada día a las 00:00 sobre un historial guardado en un archivo, simulado con el test, a partir del notebook 09.
6. **Monitoreo y política de actualización.** Cada día se medirían el RMSLE, el porcentaje de horas con subestimación mayor al 20 % (23.2 % en el test), el PSI de la demanda y de los lags, y cuántos lags faltan. La política sería reentrenar cada mes (bajó el RMSLE 1.0 % en la simulación), usar un modelo sin lags de respaldo cuando falten lags (0.4549 en validación con LightGBM sin lags) y revisar a mano los eventos extremos. Los umbrales de alarma, por ejemplo un PSI sostenido sobre 0.25 o varios días seguidos con un RMSLE diario sobre el P90 de la simulación (0.475), habría que calibrarlos. Se podría simular la política sobre el test y ver si Sandy dispara las alarmas.

7. **Pruebas automáticas.** Convertir los `assert` de los notebooks en pruebas sobre `data_loader`, `features`, `evaluate` y `predict`, y ejecutar los notebooks 07 y 08 en cada cambio. Estaría terminado cuando una ejecución automática reproduzca el RMSLE de test de 0.3505.

### 13.3 Orden sugerido

1. Primero lo básico: seguimiento y versionado (línea 4), modelo de respaldo y monitoreo (línea 6) y pruebas (línea 7). Se necesita antes de cualquier modelo nuevo.
2. Después los modelos: red secuencial y embeddings (líneas 1 y 2), comparados con LightGBM.
3. Luego la búsqueda de días parecidos y el asistente documental (línea 3).
4. Al final la integración: demostración de punta a punta y política de actualización simulada (líneas 5 y 6).

### 13.4 Riesgos y decisiones del grupo

- Un modelo profundo puede no superar al gradient boosting con 13 mil filas. Se evaluaría con la misma regla y se reportaría el resultado, sea cual sea.
- Las herramientas de MLOps no están instaladas ni comprobadas con Python 3.13.
- No hay pronósticos reales de 2012 ni datos por estación. Calibrar el ruido del clima y predecir por estación requieren datos nuevos.
- El grupo debe decidir el alcance de la etapa, las herramientas, los umbrales de monitoreo y las fechas.

## 14. Reproducibilidad

El proyecto usa Python 3.13.11 con las versiones fijadas en `requirements.txt` (`RANDOM_STATE = 42`). El README explica el orden de los notebooks; para obtener el modelo final y su validación basta con ejecutar los notebooks 07 y 08. La reproducción se verificó desde cero en una copia limpia del repositorio con un entorno nuevo, con el mismo RMSLE de test (0.3505) y predicciones idénticas.

## Referencias


- Fanaee-T, H. y Gama, J. (2013). *Event labeling combining ensemble detectors and background knowledge*. Progress in Artificial Intelligence, 1–15. doi:10.1007/s13748-013-0040-3.
- Registro de decisiones y pruebas: `reports/decisiones.md` (fichas 1 a 24). Resúmenes por sprint y notas de investigación: `docs/sprints/`.